6-3 · Producer / Consumer 애플리케이션 · 이론 1H

Producer / Consumer API와 오프셋 커밋

보내기는 왜 비동기인지, 커밋은 무엇을 보장하는지

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1시간 뒤에는

① produce() 가 버퍼에 넣을 뿐이고 flush() 가 왜 필요한지 설명한다

② delivery report 로 건별 성공을 확인하는 흐름을 안다

③ acks 0 · 1 · all 이 「어디까지 확인하나」를 정한다고 말할 수 있다

④ group.id · auto.offset.reset · 오프셋 커밋의 역할을 구별한다

다음 2.5H 실습에서 flush 를 일부러 빼고 유실을 눈으로 확인합니다

1. Producer

produce() 는 우체통에 넣는 것 — 배달은 나중

produce()	배치가 차거나	브로커로 전송	delivery report
내부 버퍼에 쌓임 즉시 반환	linger.ms 가 지나면	묶어서 한 번에	건별 콜백 성공 / 실패

flush() 없이 프로그램이 끝나면 버퍼에 남은 것이 그대로 사라진다

우체통에 넣으면 집배원이 정해진 시간에 걷어간다 — 배달 확인서가 delivery report

1. Producer

꼭 부르는 것 셋 — 그리고 왜 모아서 보내나

1. **delivery report**
   메시지별 전송 성공·실패를 알려 주는 콜백 — 등기우편의 배달 확인서
2. **flush()**
   버퍼에 남은 메시지를 다 보낼 때까지 기다린다 — 종료 직전에 반드시
3. **poll(0)**
   쌓인 콜백을 처리하라는 신호 — 루프 안에서 주기적으로
4. **왜 모아서 보내나**
   한 건씩 보내면 네트워크 왕복이 너무 많다 — 묶어 보내는 쪽이 훨씬 빠르다

> **flush() 를 빠뜨리는 것이 이 주차에서 가장 흔한 실수 — 오류 없이 정상 종료되는데 버퍼에 남은 것이 사라진다(실습 실측 1000 건 중 0 건). try/finally 로 감싸 두세요**

acks — 어디까지 확인하고 성공으로 볼 것인가

acks	언제 성공으로 보나	속도	안전
0	보내자마자 (확인 안 함)	가장 빠름	유실 가능
1 (기본)	리더 브로커가 받으면	중간	리더가 죽으면 유실 가능
all	복제본까지 다 받으면	가장 느림	가장 안전

단일 브로커 실습에서는 차이가 잘 안 보인다 — 운영에서는 「돈이 오가는 이벤트는 all」 같은 기준으로

2. Consumer

읽고, 어디까지 읽었는지 기록한다

1 **group.id — 어느 그룹인가**
같은 그룹끼리는 파티션을 나눠 맡고, 그룹이 다르면 각자 전체를 다 읽는다

2 **auto.offset.reset — 처음엔 어디부터**
earliest(처음부터) / latest(지금부터) — 오프셋이 아직 없는 그룹에만 적용

3 **오프셋 커밋 — 「여기까지 읽었다」**
Kafka 에 기록해 두면 재시작해도 이어서 읽는다

poll(timeout)

없으면 기다렸다

None 반환

> **이미 읽은 적 있는 그룹에 earliest 를 줘도 처음부터 다시 읽지 않는다 — 그때는 6-2 의 --reset-offsets**

2. Consumer

자동 커밋과 수동 커밋 — 무엇을 잃고 무엇을 얻나

자동 커밋 (기본)

5초마다 자동 — 코드는 단순하지만 처리 실패해도 커밋될 수 있다

- 받고 처리 중인데 커밋 시각이 온다
- 실패해도 「읽었다」로 기록 → 유실
- 최대 한 번(at-most-once) 쪽

VS

수동 커밋

내가 commit() 부를 때 — 처리 완료 후에 커밋할 수 있다

- 처리를 끝낸 뒤 커밋 → 유실 없음
- 커밋 직전에 죽으면 같은 것을 또 읽음 → 중복
- 최소 한 번(at-least-once) 쪽

2. Consumer

유실을 막으면 중복이 남는다 — 그래서 멱등

1. **자동 커밋은 유실 쪽**
   처리에 실패해도 「읽었다」가 기록될 수 있다
2. **수동 커밋은 중복 쪽**
   유실은 없지만 커밋 직전에 죽으면 같은 메시지를 다시 읽는다
3. **그래서 받는 쪽이 멱등해야 한다**
   4-3 의 MERGE, 2-5 의 UNIQUE 제약이 그 역할 — 두 번 받아도 결과가 같게

exactly-once 는

트랜잭션으로

가능·복잡·느림

> 대부분은 「최소 한 번 + 멱등한 수신」으로 간다

이 과정을 관통하는 원칙

중복은 막는 것이 아니라 견디는 것

01

2-5 · INSERT IGNORE

크롤러가 같은 글을

두 번 저장해도 한 행

02

4-3 · MERGE

같은 키가 또 오면

덮어쓰거나 건너뛴다

03

6-3 · 멱등한 수신

커밋 직전에 죽어

다시 읽어도 결과 같게

세 가지가 전부 같은 이야기 — 재시도가 안전하려면 받는 쪽이 멱등해야 한다

정리

이 1시간에서 꼭 가져갈 것

1. produce() 는 버퍼에 넣을 뿐이다 — flush() 없이 끝내면 사라진다
2. delivery report 로 건별 성공을 확인한다
3. acks 는 「어디까지 확인하고 성공으로 볼지」를 정한다
4. auto.offset.reset 은 오프셋이 없는 그룹에만 적용된다
5. 수동 커밋 + 멱등한 수신이 실무의 기본 조합이다

6-3 · 이론 1H 마무리

다음은 Producer / Consumer 코드 실습 (2.5H)

파이썬으로 양쪽을 직접 만들고, flush 를 빼 유실을, 수동 커밋으로 중복을, 그리고 LAG 을 눈으로 확인합니다

- ✓ **producer.py — 주문 이벤트 30건 발행**
- ✓ **flush 없이 1000건 보내 유실 재현**
- ✓ **consumer.py — 고객별 누적 집계**
- ✓ **수동 커밋 · LAG 관찰**

준비할 것

실습점검 — compose 켜고 orders 비우기